

# Lecture 04: Classes and Objects

## Introduction to Programming with Java

# Today's learning goals

By the end of this lecture you should be able to:

1. Distinguish a **class** (blueprint) from an **object** (instance)
2. Declare fields and simple constructors
3. Create objects with `new` and call instance methods
4. Explain how an object holds its own state

# Agenda

1. From methods to objects
2. Classes as blueprints
3. Fields (instance variables)
4. Constructors
5. Creating and using objects
6. Multiple objects
7. A small worked example

So far: data (variables/arrays) and methods often live separately in `main`. Real programs group **data + behaviour** that belong together:

- A bank account: balance + deposit/withdraw
- A student: name, id + print summary
- A game character: health, position + move/takeDamage

# Class vs object

## Class

Blueprint / type

What kind of thing?

Which fields and methods?

## Object (instance)

A concrete example

Created with `new`

Has its own field values

Analogy: “Dog” is the class; “Rex” and “Bella” are objects.

# You already used objects

- `String s = "Hi";` — `s` refers to a `String` object
- `s.length()` — instance method on that object
- Arrays are objects too (`scores.length`)

Today we define **our own** classes.

# A minimal class

```
1 public class Student {  
2     String name;    // field (instance variable)  
3     int id;         // field  
4 }
```

Fields store the object's state. Each Student object has its own name and id.

# Creating an object

```
1 Student a = new Student();  
2 a.name = "Ada";  
3 a.id = 1001;  
4  
5 Student b = new Student();  
6 b.name = "Alan";  
7 b.id = 1002;  
8  
9 System.out.println(a.name);    // Ada  
10 System.out.println(b.name);    // Alan
```

## `new` does the work

1. Allocates memory for a new object
2. Runs a constructor (next slides)
3. Returns a **reference** you store in a variable

### Common mistake

Thinking `Student a;` already creates an object. That only declares a variable; use `new` to create the instance.

## Quiz 1 — Class vs object

### Quiz

Which statement is correct?

1. A class is one running program; an object is a file
2. A class is a blueprint; an object is a concrete instance created with `new`
3. Objects cannot hold data
4. `new` deletes a class

*Pause — answer (clicker / raise hand / neighbour) before the next slide.*

## Answer — Quiz 1

### Answer

2. Class = blueprint; object = instance created with `new`.

# What is a constructor?

A special method-like member that runs when you use `new`.

- Same name as the class
- No return type (not even `void`)
- Initialises fields to a sensible starting state

# Constructor example

```
1 public class Student {  
2     String name;  
3     int id;  
4  
5     public Student(String name, int id) {  
6         this.name = name;  
7         this.id = id;  
8     }  
9 }
```

`this.name` means “the field”, when the parameter has the same name.

# Using the constructor

```
1 Student a = new Student("Ada", 1001);  
2 Student b = new Student("Alan", 1002);  
3 System.out.println(a.name + " #" + a.id);
```

Prefer constructors over leaving fields uninitialised and setting them later ad hoc.

# Default constructor reminder

If you write **no** constructor, Java provides a no-arg default (fields get default values: 0, null, false, ...). If you write *any* constructor, the automatic no-arg one disappears unless you add it yourself.

## Quiz 2 — Constructors

### Quiz

Which is true of a constructor?

1. It must return `int`
2. Its name must match the class name and it has no return type
3. It is called with `Student.constructor()`
4. It replaces the need for `main`

*Pause — answer (clicker / raise hand / neighbour) before the next slide.*

### Answer

2. Same name as the class; no return type; invoked via `new`.

# Behaviour belongs with the data

```
1 public class Student {  
2     String name;  
3     int id;  
4  
5     public Student(String name, int id) {  
6         this.name = name;  
7         this.id = id;  
8     }  
9  
10    public void printSummary() {  
11        System.out.println(name + " (id " + id + ")");  
12    }  
13 }
```

# Calling instance methods

```
1 Student a = new Student("Ada", 1001);  
2 a.printSummary();    // Ada (id 1001)
```

Notice: not static. The method runs **on** a particular object and can use that object's fields.

# Methods can return values

```
1 public class BankAccount {  
2     private double balance; // preview: next lecture  
3  
4     public BankAccount(double start) {  
5         balance = start;  
6     }  
7  
8     public void deposit(double amount) {  
9         balance += amount;  
10    }  
11  
12    public double getBalance() {  
13        return balance;  
14    }  
15 }
```

# Each object has its own state

- Two `Student` variables can refer to two different objects
- Changing `a.name` does not change `b.name`
- Two variables can also refer to the *same* object (aliases)

# References (light)

```
1 Student a = new Student("Ada", 1);  
2 Student b = a;           // b refers to the same object  
3 b.name = "Augusta";  
4 System.out.println(a.name); // Augusta
```

## Common mistake

Assuming = always copies all field values. For objects, = copies the **reference**.

## Quiz 3 — What prints?

```
1 Student a = new Student("Ada", 1);  
2 Student b = new Student("Alan", 2);  
3 b = a;  
4 System.out.println(b.name);
```

Quiz

Output?

1. Alan
2. Ada
3. null
4. Compile error

*Pause — answer (clicker / raise hand / neighbour) before the next slide.*

## Answer — Quiz 3

### Answer

2. Ada — after `b = a`, both refer to Ada's object.

# Counter class

```
1 public class Counter {  
2     int value;  
3  
4     public Counter() {  
5         value = 0;  
6     }  
7  
8     public void hit() {  
9         value++;  
10    }  
11  
12    public int getValue() {  
13        return value;  
14    }  
15 }
```

# Using Counter

```
1 Counter c1 = new Counter();  
2 Counter c2 = new Counter();  
3 c1.hit();  
4 c1.hit();  
5 c2.hit();  
6 System.out.println(c1.getValue()); // 2  
7 System.out.println(c2.getValue()); // 1
```

## File layout (typical beginner setup)

- One public class per file; file name matches class name
- `Student.java`, `Counter.java`, ...
- A class with `main` to try things (`Main.java` or similar)

### Try this

Create `Rectangle` with fields `width`, `height`, a constructor, and method `area()`.

# Key takeaways

1. Class = blueprint; object = instance via `new`
2. Fields hold per-object state
3. Constructors initialise new objects
4. Instance methods operate on a particular object's data
5. Assignment copies references for objects

# Coming next

## Lecture 05

Encapsulation: `public` / `private`, getters/setters

Light intro to `static` vs instance members

# Questions?

Fields, constructors, or references—where should we slow down?